∀ ∃ ∧ ∨ →

Can a program decide whether another program will ever stop? How do you prove that a function is correct for every input, not just for the ones you tested? How does a computer find a proof on its own? This book answers these questions with precise definitions, complete proofs, and working TypeScript programs.

Every central idea becomes code. You implement a formula evaluator, the conversion to conjunctive normal form, the Davis-Putnam SAT solver, a backtracking constraint solver, and the unification algorithm. Then you use industrial tools, the SAT solver MiniSat, Microsoft's Z3, and the theorem provers Vampire, Prover9, and Mace4, to see how far the same ideas reach.

Topics include

- correctness proofs by computational induction and symbolic execution
- propositional logic, normal forms, and the Davis-Putnam algorithm
- first-order logic and constraint programming
- automated theorem proving with resolution and unification

Along the way you place eight queens on a chessboard, solve the Zebra Puzzle and a Sudoku, lead a knight across all 64 squares, and find out who killed Aunt Agatha.

Written for first-year students of computer science and for every programmer who wants to reason precisely about software. All programs are available as Jupyter notebooks at github.com/karlstroetmann/Logic.

Karl Stroetmann is a professor at the Baden-Wuerttemberg Cooperative State University (DHBW), where this book grew out of a course on theoretical computer science. Until 2002 the author worked on hardware verification at the Siemens Research Center.

Theoretical Computer Science Logic & Puzzles TypeScript Edition Karl Stroetmann

∀ ∃ ∧ ∨ →

Theoretical Computer Science

Logic & Puzzles



TypeScript Edition